CS313 Lab Report

Betty Liu u2061245

April 4, 2024

Contents

In this assignment, I used ChatGPT to help me check grammar and correct wrong spellings.

Part I
Lab 1

1 Task 1-2

1.1 Command to create the hello world package

Inside ros_ws create src file, than create python package named r2

```
mkdir src && cd src
ros2 pkg create --build-type ament_python r2
```

For these 2 lines. Create folder named src, and create a project named r2 that mainly using python.

Add python file

```
cd r2/r2
vim printer.py
```

1.2 Code snippet showing the hello world program

What inside printer.py

```
#! /usr/bin/python
import rclpy
from rclpy.node import Node

rclpy.init(args=None)
node = Node('printer_node')  # Note that there should have no
space in node name
node.get_logger().info('HELLO WORLD - ROS2')
rclpy.spin(node)
node.destroy_node()
rclpy.shutdown()
```

Import rclpy, then create a node named printer_node. Then create an info message HELLO WORLD - ROS2 into log of the node. rclpy.spin(node) keep node running until it shutdown. node.destroy_node() and rclpy.shutdown() destroy the node and shutdown tclpy.

continue for 1.1, modify setup.py

```
cd ..
```

in section field entry_points, add

```
    'console_scripts': [
      'prt = r2.printer:main'
    ],
```

setuptools generate a command prt, when user input prt it runs printer.py in r2 module. In such cases, we don't need to explicitly define a main function. By default, the printer file will be executed starting from the top.

1.3 Commands to run the program with ros2 run and ros2 launch

For ros2 run

At ros_ws

```
colcon build --packages-select r2
source install/local_setup.bash
ros2 run r2 prt
```

Setup environment variables. Using ros2 run to run a single node. In this case, it runs executable named prt in package r2.

For ros2 launch

At ros_ws

```
mkdir launch
vim launch/node_launcher.py
```

Edit launcher file

```
#! /usr/bin/python
import launch
import launch_ros.actions

def generate_launch_description():
  return launch.LaunchDescription([
    launch_ros.actions.Node(
      package='r2',
      executable='prt',
      output='screen',
```

```
      name='custom_py_node_name'
    )
  ])
```

This code defines a launch file for launching a ROS node and execute prt command, belonging to the r2 package. Then specifies that the node's output will be printed to the screen.

Configure environment

```
source install/local_setup.bash
ros2 launch launch/node_launcher.py
```

Using launch command and read node and parameters given by node_launcher.py

2 Task 1-3

2.1 Snippet of your pubsub_launcher.launch file

I'm using yaml format, this file is under ros_ws/src/r2/launch. File name is pubsub_launcher.launch.yaml

```
launch:

- node:
    pkg: "r2"
    exec: "pub"
    name: "publisher"

- node:
    pkg: "r2"
    exec: "sub"
    name: "subscriber"
```

Define two nodes, named publisher and subscriber, both belonging to the r2 package, and use the pub and sub commands, respectively.

Add line in setup.py, now the setup.py is

```
import os
from glob import glob
from setuptools import setup

package_name = 'r2'

setup(
```

```
    name=package_name,
    version='0.0.0',
    packages=[package_name],
    install_requires=['setuptools'],
    zip_safe=True,
    maintainer='lucia',
    maintainer_email='acyanbird@gmail.com',
    description='TODO: Package description',
    license='TODO: License declaration',
    tests_require=['pytest'],
    entry_points={
        'console_scripts': [
            'prt = r2.printer:main',
            'pub = r2.publisher:main',
            'sub = r2.subscriber:main'
        ],
    },
    data_files=[
        # ... Other data files
        # Include all launch files.
        (os.path.join('share', package_name, 'launch'),
        glob(os.path.join('launch', '*launch.[pxy][yma]*')))]
)
```

Comparing to previous setup file, it generate 2 more command pub and sub. They will run main function of publisher.py and subscriber.py

data_file means it will include all the file end with launch.py or launch.xml or launch.yaml under folder launch in package. Sowe can call it under r2.

2.2 Commands to launch it with ros2 launch

Then compile and install them, go to ros_ws

```
colcon build --packages-select r2
source install/local_setup.bash
ros2 launch r2 pubsub_launcher.launch.yaml
```

Build package r2, install environment variables and using yaml launch file.

2.3 Screenshot of your output

```
lucia@bluejay-ubuntu:~/robot/ros_ws/src/r2/launch$ ros2 launch r2 pubsub_launcher.launch.yaml
[INFO] [launch]: All log files can be found below /home/lucia/.ros/log/2024-02-03-18-41-47-051366-bluejay-ubuntu-20683
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [pub-1]: process started with pid [20685]
[INFO] [sub-2]: process started with pid [20687]
[pub-1] [INFO] [1706985707.975869820] [publisher]: Publishing: "Hello World: 0"
[sub-2] [INFO] [1706985707.977400117] [subscriber]: I heard: "Hello World: 0"
[pub-1] [INFO] [1706985708.454612722] [publisher]: Publishing: "Hello World: 1"
[sub-2] [INFO] [1706985708.455567790] [subscriber]: I heard: "Hello World: 1"
[pub-1] [INFO] [1706985708.954173778] [publisher]: Publishing: "Hello World: 2"
[sub-2] [INFO] [1706985708.954525785] [subscriber]: I heard: "Hello World: 2"
[pub-1] [INFO] [1706985709.454338135] [publisher]: Publishing: "Hello World: 3"
[sub-2] [INFO] [1706985709.454787430] [subscriber]: I heard: "Hello World: 3"
[pub-1] [INFO] [1706985709.954775001] [publisher]: Publishing: "Hello World: 4"
[sub-2] [INFO] [1706985709.955556186] [subscriber]: I heard: "Hello World: 4"
```

We create 2 nodes publisher and subscriber, publisher create message hello world with increasingorder number each 0.5 second. Then subscriber get the message and echo them.

Part II
Lab 2

3 Task 2-1:

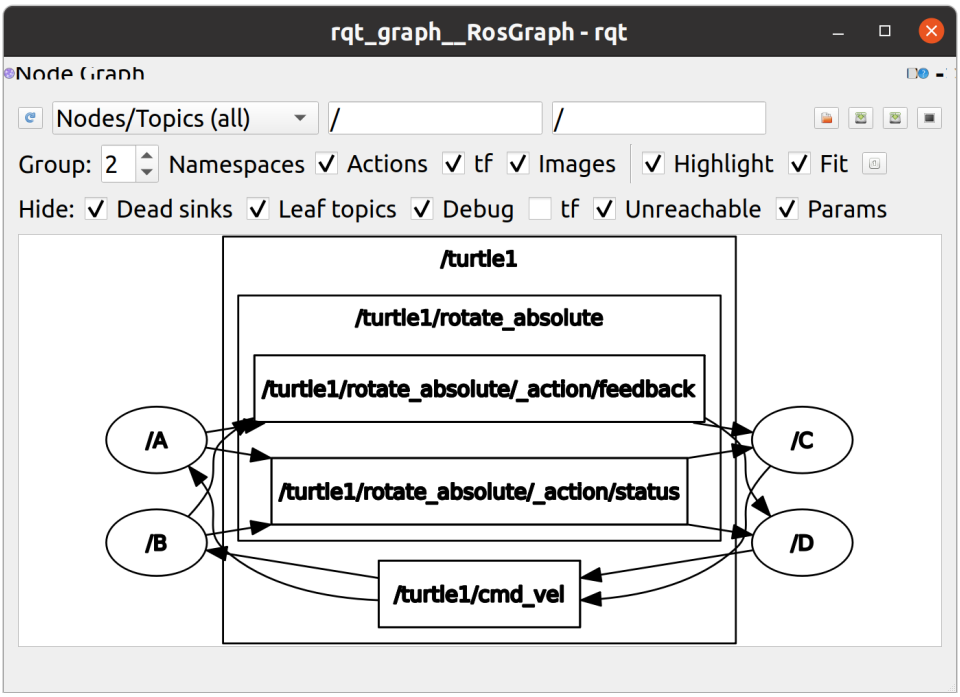
Visualisation of the four nodes: turtlesim_node (A and B) and turtle_teleop_key (C and D) with rqt_graph

Run these commands in different terminals because we need them run simultaneously. Remap is used to pass argument, which is node name to each node.

```
ros2 run turtlesim turtlesim_node --ros-args --remap __node:=A
ros2 run turtlesim turtlesim_node --ros-args --remap __node:=B

ros2 run turtlesim turtle_teleop_key --ros-args --remap
__name:=C
ros2 run turtlesim turtle_teleop_key --ros-args --remap
__name:=D

rqt_graph
```



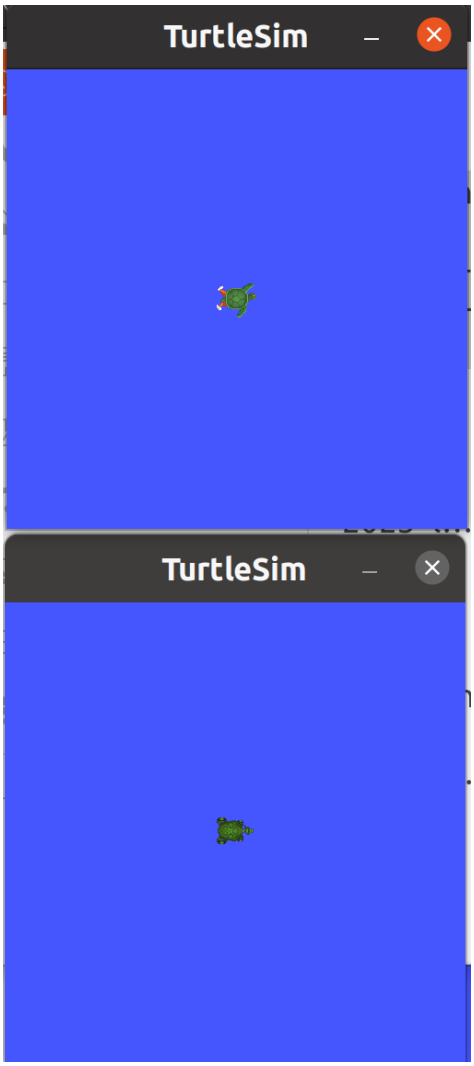
4 Task 2-2

4.1 Commands to teleport the turtlebot to a specified location and the screenshot of the result

Command to reset turtle

```
ros2 service call /reset std_srvs/srv/Empty
```

Turle status



Teleport using relative

```
ros2 service call /turtle1/teleport_relative
turtlesim/srv/TeleportRelative "{linear: 1}"
```

And get

4.2 Commands to change the background color to red using ros2 param and the screenshot of the result

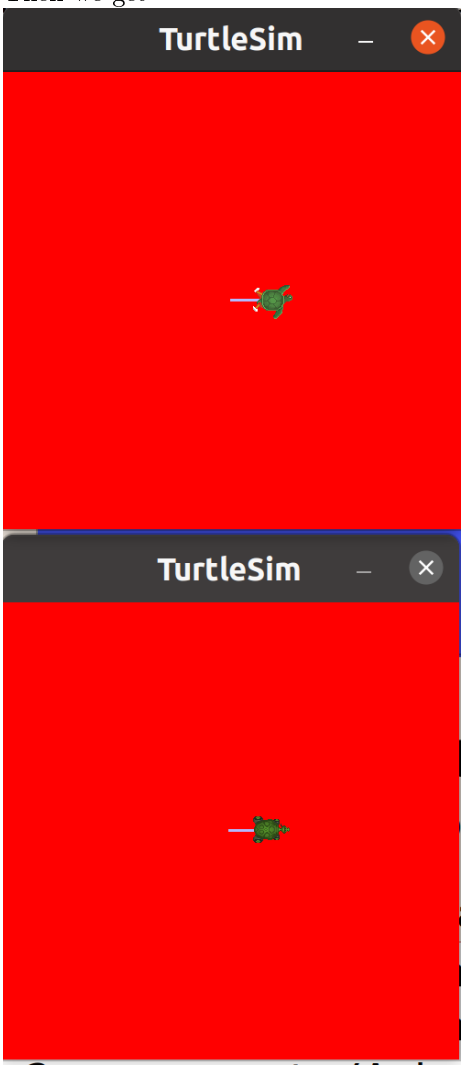
Set node A and B seperately

```
ros2 param set /A background_b 0
ros2 param set /A background_g 0
ros2 param set /A background_r 255

ros2 param set /B background_b 0
ros2 param set /B background_g 0
```

```
ros2 param set /B background_r 255
```

Then we get

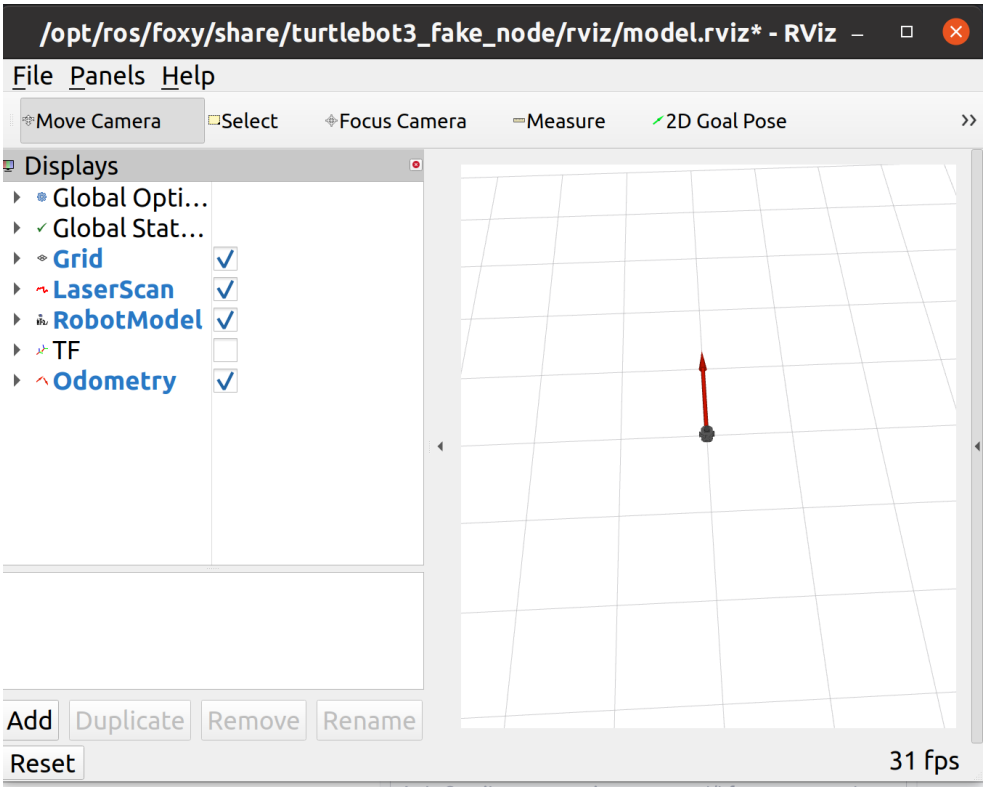


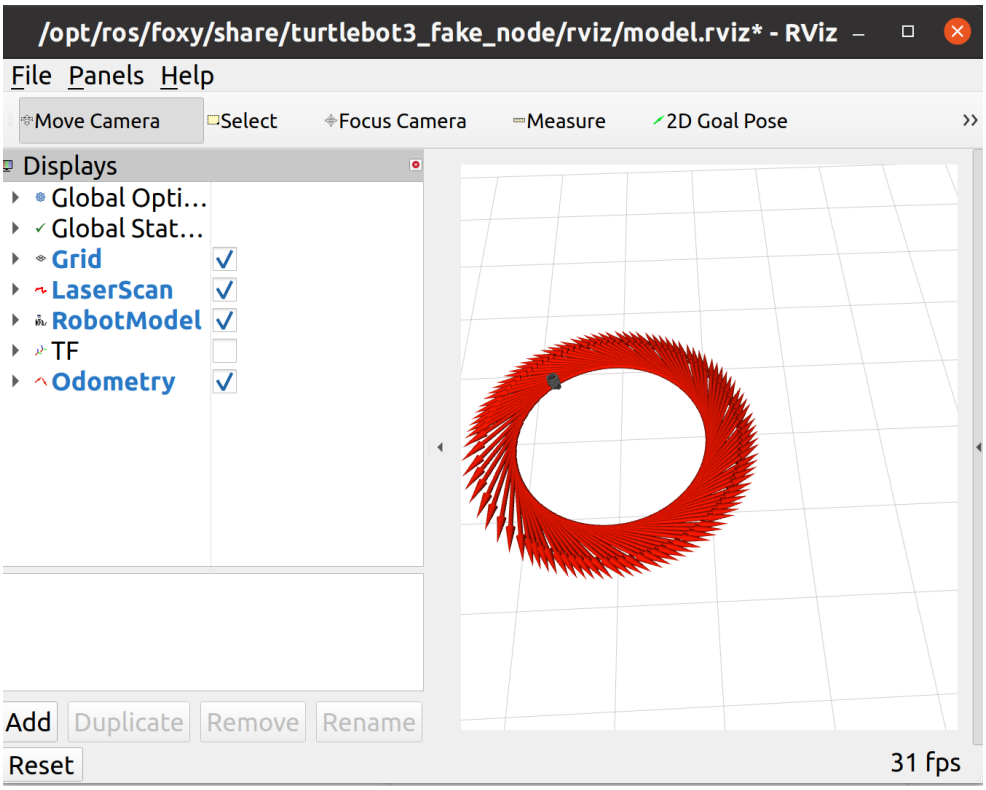
5 Task 2-3

5.1 Code snippet to drive the turtlebot in circle with screenshot from RViz

Keep the virtual Turtlebot in RViz windows.

```
ros2 launch turtlebot3_fake_node turtlebot3_fake_node.launch.py
```





Code

For trajectory.py

```
#! /usr/bin/env python
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

class Mover(Node):
    def __init__(self):
        super().__init__('vel_publisher')
        self.publisher_ = self.create_publisher(Twist,
        'cmd_vel', 10)
        self.timer_ = self.create_timer(0.5,
        self.timer_callback)
        self.i = 0
        self.get_logger().info("Press CTRL + C to terminate")

    def timer_callback(self):
        msg = Twist()
        # Set the velocity here: using Twist.linear.x/y/z and/or
        Twist.angular.x/y/z
```

```
        msg.linear.x = 2.0
        msg.angular.z = 1.8

        self.publisher_.publish(msg)
        self.get_logger().info('Publishing: "%s"' % msg)

def main(args=None):
    rclpy.init(args=args)
    mover = Mover()
    rclpy.spin(mover)
    mover.destroy_node()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

For setup.py, modify entry_points section only

```
entry_points={
        'console_scripts': [
            'move = move_turtlebot3.trajectory:main',
        ],
    },
```

5.2 Clearly describe your approach and explain your code

Firstly, create a package named move_turtlebot3

```
ros2 pkg create --build-type ament_python move_turtlebot3
```

Then fill in the skeleton provided by lab, named trajectory.py in same level of __init__.py

Firstly, the program run the main function. It instantiate mover from class Mover. Mover class is initialized with a publisher that publishes messages of type Twist to the cmd_vel topic.

timer_callback publishes messages of type Twist to the cmd_vel topic every 0.5 second using create_timer function. I changed the default value so it could move faster. Set linear to 2 and angular to 1.8

After Ctrl+C the process end.

```
colcon build --packages-select move_turtlebot3
source install/setup.bash
ros2 run move_turtlebot3 move
```

These commands run the robot.

Part III
Lab 3

Launch turtlebot3 in gazbo first

```
ros2 launch turtlebot3_gazebo empty_world.launch.py
```

6 Task 3-1:

6.1 Code snippet to drive the turtlebot to move the square shape trajectory

Create project

```
ros2 pkg create --build-type ament_python gazebo_square
```

I create 2 nodes, one to move robot, the other one use to record the position
For position record

```
import os
import time

import rclpy
from rclpy.node import Node
from nav_msgs.msg import Odometry
import threading


class OdomSubscriber(Node):
    def __init__(self):
        super().__init__('odom_subscriber')
        self.subscription = self.create_subscription(
            Odometry,
            'odom',
            self.listener_callback,
            10)

    def listener_callback(self, msg):
        position = msg.pose.pose.position
```

```
        with open('position.txt', 'a') as f:
            f.write(f'x: {position.x}, y: {position.y}\n')
        rclpy.shutdown()


def main(args=None):
    rclpy.init(args=args)
    odom_subscriber = OdomSubscriber()
    t = threading.Thread(target=rclpy.spin,
    args=(odom_subscriber,), daemon=True)
    t.start()
    time.sleep(1)
    os._exit(0)


if __name__ == '__main__':
    main()
```

For robot movement, it also call position record whithin the program

```
import subprocess
import rclpy
from geometry_msgs.msg import Twist
from math import pi
import threading

distance = 4.0
angle = pi / 2.0


def main(args=None):
    rclpy.init(args=args)
    rate_node = rclpy.create_node('rate_node')

    t = threading.Thread(target=rclpy.spin, args=(rate_node,),
    daemon=True)  # the rate rely on the spin function
    # running in a separate thread
    t.start()

    frq = 10  # frequency: Hz
    fwd_time = 8
    rot_time = 5

    rate = rate_node.create_rate(frq)  # create a rate object
    with 10Hz, use rate.sleep() to keep the frequency
```

```
    pub = rate_node.create_publisher(Twist, 'cmd_vel', 10)
    try:
        msg = Twist()
        subprocess.run(["ros2", "service", "call",
        "/reset_simulation", "std_srvs/srv/Empty"])  # reset the
        simulation
        subprocess.run(['ros2', 'run', 'gazebo_square',
        'position'])
        for i in range(4 * fwd_time * frq):
            msg.linear.x = distance / fwd_time
            msg.angular.z = 0.0
            pub.publish(msg)    # publish the message

            rate_node.get_logger().info('[Translation]
            Publishing: "%s"' % i)
            rate.sleep()  # the code will sleep to keep the
            frequency, in this case 10Hz

            if (i + 1) % (fwd_time * frq) == 0:
                rate.sleep()
                subprocess.run(['ros2', 'run', 'gazebo_square',
                'position'])
                for _ in range(rot_time * frq):
                    msg.linear.x = 0.0
                    msg.angular.z = angle / rot_time
                    pub.publish(msg)

                    rate_node.get_logger().info('[Rotation]
                    Publishing: "%s"' % i)
                    rate.sleep()

        msg.linear.x = 0.0
        msg.angular.z = 0.0
        pub.publish(msg)
        rate_node.get_logger().info('[Stop] Publishing: "%s"' %
        i)
        with open('position.txt', 'a') as f:
            f.write("after finished the square\n")
        subprocess.run(['ros2', 'run', 'gazebo_square',
        'position'])

    except KeyboardInterrupt:
        pass

    rate_node.destroy_node()
```

```
    rclpy.shutdown()
    t.join()


if __name__ == '__main__':
    main()
```

Then add the entry point at setup.py

```
entry_points={
        'console_scripts': [
            'move=gazebo_square.trajectory:main',
            'position=gazebo_square.record_position:main',
        ],
```

Build and run the project

```
colcon build --packages-select gazebo_square
source install/setup.bash
ros2 run gazebo_square move
```

6.2 Clearly describe your approach and explain your code

In trajectory.py, I utilized the provided structure from the lab. First, it resets the simulation. Then, the robot moves forward 4 meters and turns 90 degrees, repeating this process 4 times.

To enhance accuracy, I slowed down the speed. I adjusted msg.linear.x to distance / fwd_time to slow down the speed to 0.5m/s. When it reaches the forward time (i increments by 100 per second), it stops and starts turning. After executing 4 times, the loop stops, and the robot speed and rotation are set to 0 again.

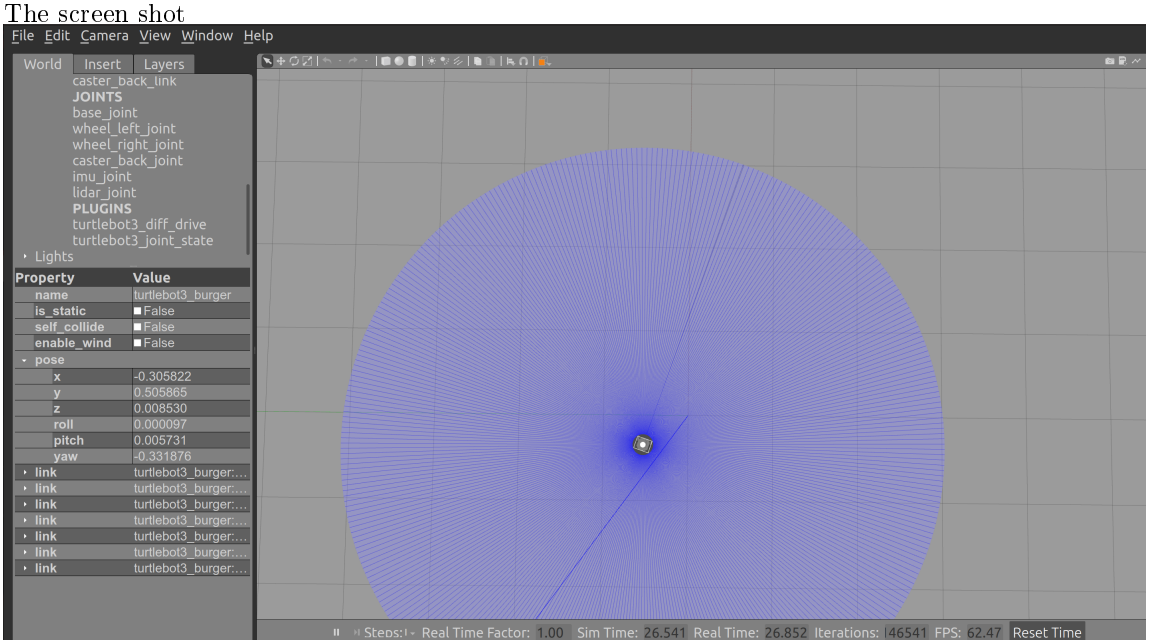
For position recording, the node subscribes to the odom topic and retrieves the pose once, adding it to the position.txt file. It creates a thread to execute, waits for 0.2 seconds, and exits. It only records the position once, right after writing into the txt file with rclpy.shutdown(). trajectory.py calls this node every time it needs to record the position.

6.3 Screenshots showing that your robot can visit the four waypoints (with reasonable errors, e.g. <1m).

Final output in position.txt

```
x: 7.382641119197687e-05, y: -3.2001355556987805e-07
x: 4.320247894872797, y: 0.018901195616114114
x: 4.7065540745287615, y: 4.216054346164607
x: 0.520388013529021, y: 4.875602328810379
x: -0.27317267205084467, y: 0.7377587102866461
after finished the square
x: -0.30583353582438483, y: 0.5058357508299174
```

The screen shot



7 Task 3-2:

7.1 Code to simulate motion noise and explain how it works

Create project

```
ros2 pkg create --build-type ament_python lab32
```

Use most of the codes from previous task but import parameters

```
import subprocess
import rclpy
from geometry_msgs.msg import Twist
```

```
from math import pi
import threading
import numpy as np

distance = 4.0
angle = pi / 2.0


def main(args=None):
    rclpy.init(args=args)
    rate_node = rclpy.create_node('rate_node')
    rate_node.declare_parameter('l_noise', 0.0)
    rate_node.declare_parameter('a_noise', 0.0)
    rate_node.declare_parameter('file_name', 'position.txt')

    linear_noise =
    rate_node.get_parameter('l_noise').get_parameter_value().double_value
    angular_noise =
    rate_node.get_parameter('a_noise').get_parameter_value().double_value
    file_name =
    rate_node.get_parameter('file_name').get_parameter_value().string_value

    t = threading.Thread(target=rclpy.spin, args=(rate_node,),
    daemon=True)  # the rate rely on the spin function
    # running in a separate thread
    t.start()

    frq = 10  # frequency: Hz
    fwd_time = 8
    rot_time = 5

    rate = rate_node.create_rate(frq)  # create a rate object
    with 10Hz, use rate.sleep() to keep the frequency
    pub = rate_node.create_publisher(Twist, 'cmd_vel', 10)
    try:
        msg = Twist()
        subprocess.run(["ros2", "service", "call",
        "/reset_simulation", "std_srvs/srv/Empty"])  # reset the
        simulation
        for i in range(4 * fwd_time * frq):
            noise = np.random.normal(0, linear_noise)
            msg.linear.x = distance / fwd_time + noise
            msg.angular.z = 0.0
            pub.publish(msg)    # publish the message

```

```
            rate_node.get_logger().info('[Translation] Publishing: "%s"' % i)
            rate.sleep()  # the code will sleep to keep the frequency, in this case 10Hz

            if (i + 1) % (fwd_time * frq) == 0:
                rate.sleep()
                for _ in range(rot_time * frq):
                    noise = np.random.normal(0, angular_noise)
                    msg.linear.x = 0.0
                    msg.angular.z = angle / rot_time + noise
                    pub.publish(msg)

                    rate_node.get_logger().info('[Rotation] Publishing: "%s"' % i)
                    rate.sleep()

        msg.linear.x = 0.0
        msg.angular.z = 0.0
        pub.publish(msg)
        rate_node.get_logger().info('[Stop] Publishing: "%s"' % i)
        subprocess.run(['ros2', 'run', 'lab32', 'position', '--ros-args', '-p', f'file_name:={file_name}'])

    except KeyboardInterrupt:
        pass

    rate_node.destroy_node()
    rclpy.shutdown()
    t.join()


if __name__ == '__main__':
    main()
```

Reset simulation at beginning of each turn. Using rate_node.declare_parameter to declare linear noise, angular noise and file name to record the position. At each time sending twsit, using numpy to calculate noise randomly. Only record position when the robot finshied each iteration and stopped.

For position

```
import os
import time

```

```python
import rclpy
from rclpy.node import Node
from nav_msgs.msg import Odometry
import threading


class OdomSubscriber(Node):
    def __init__(self):
        super().__init__('odom_subscriber')
        self.subscription = self.create_subscription(
            Odometry,
            'odom',
            self.listener_callback,
            10)
        self.declare_parameter('file_name', 'position.txt')
        self.file_name =
        self.get_parameter('file_name').get_parameter_value().string_value

    def listener_callback(self, msg):
        position = msg.pose.pose.position
        with open(self.file_name, 'a') as f:
            f.write(f'{float(position.x)}
            {float(position.y)}\n')
        rclpy.shutdown()


def main(args=None):
    rclpy.init(args=args)
    odom_subscriber = OdomSubscriber()
    t = threading.Thread(target=rclpy.spin,
    args=(odom_subscriber,), daemon=True)
    t.start()
    time.sleep(0.2)
    os._exit(0)


if __name__ == '__main__':
    main()
```

Using parameter input as file name, append mode to add each input as new line of each turn's final position.

Change setup.py

```
entry_points={
        'console_scripts': [
```

```
            'move = lab32.noise:main',
            'position = lab32.position:main',
        ]
```

Build project by

```
colcon build --packages-select lab32
source install/setup.bash
```

I'm using a python file to run simulation in different noise level, totally 30 turns.

```
import subprocess

for _ in range(10):
    # run without noise
    subprocess.run(["ros2", "run", "lab32", "move",
    "--ros-args", "-p", "l_noise:=0.0", "-p", "a_noise:=0.0",
    "-p", "file_name:=p1.txt"])
    # run with low noise
    subprocess.run(["ros2", "run", "lab32", "move",
    "--ros-args", "-p", "l_noise:=0.002", "-p",
    "a_noise:=0.001", "-p", "file_name:=p2.txt"])
    # run with high noise
    subprocess.run(["ros2", "run", "lab32", "move",
    "--ros-args", "-p", "l_noise:=0.02", "-p", "a_noise:=0.01",
    "-p", "file_name:=p3.txt"])
```

Start simulation

```
ros2 launch turtlebot3_gazebo empty_world.launch.py
```

Then we get 3 ouput file record position at different noise levels.
Without noise

```
-0.07379741707830968 -0.005535579744299584
-0.09837604542784895 0.04046680427700269
-0.09998493690126975 0.039084283123288974
-0.09067430022097647 -0.006590720589686069
-0.056109222267952785 -0.004125044065856912
-0.11718038979564332 0.03818406308235062
-0.07407609777581077 0.0376400441594107
-0.1171306425950193 -0.006453745232904999
```

```
-0.09054764641489077 -0.020189422754572528
-0.11526432770937795 0.03960652185335872
```

With linear 0.002, angular 0.001

```
-0.08198766225546675 0.05629483143054131
-0.0930786812722532 -0.023340952372412252
-0.08502499757644746 0.02668814762790972
-0.06808187611482151 -0.0027894148811806263
-0.07092270258865978 -0.01882176316745092
-0.10015868913854332 -0.002877870861875369
-0.15572164463562055 0.014292587828699119
-0.09376989602721506 0.03569018505915051
-0.12468995489578315 0.002257994534304147
-0.07993997799210165 0.003451665922614623
```

With linear 0.02, angular 0.01

```
-0.12356562189431151 0.04766069078142874
-0.11720737878024452 0.02482205515361496
-0.23691176549448564 0.04915883750002336
-0.18257528463088027 0.1794309597282791
-0.06804106917224081 -0.03366920421342969
-0.17532397965288588 0.12699797425240802
-0.24430642924171028 0.11782102586375498
0.03308035010122279 -0.010423388365407172
-0.24901366347232023 0.017661109019843843
-0.09769253613025125 0.09662218913993334
```

8 Task 3-3:

8.1 Code to calculate the covariance matrix from the final locations

Using numpy read data from txt file, and calculate them

```
import numpy as np

data1 = np.loadtxt('p1.txt')
data2 = np.loadtxt('p2.txt')
data3 = np.loadtxt('p3.txt')

data1_cov = np.cov(data1, rowvar=False)
```

```
data2_cov = np.cov(data2, rowvar=False)
data3_cov = np.cov(data3, rowvar=False)

data1_std_devs = np.sqrt(np.diag(data1_cov))
data2_std_devs = np.sqrt(np.diag(data2_cov))
data3_std_devs = np.sqrt(np.diag(data3_cov))


print("p1 cov: \n", np.cov(data1, rowvar=False))
print("p2 cov: \n", np.cov(data2, rowvar=False))
print("p3 cov: \n", np.cov(data3, rowvar=False))

print("p1 std devs: ", data1_std_devs)
print("p2 std devs: ", data2_std_devs)
print("p3 std devs: ", data3_std_devs)

# Calculate the standard deviations for each column
std_dev1_x = np.std(data1[:, 0])
std_dev1_y = np.std(data1[:, 1])

std_dev2_x = np.std(data2[:, 0])
std_dev2_y = np.std(data2[:, 1])

std_dev3_x = np.std(data3[:, 0])
std_dev3_y = np.std(data3[:, 1])

# Print the standard deviations
print("p1 std dev x: ", std_dev1_x)
print("p1 std dev y: ", std_dev1_y)

print("p2 std dev x: ", std_dev2_x)
print("p2 std dev y: ", std_dev2_y)

print("p3 std dev x: ", std_dev3_x)
print("p3 std dev y: ", std_dev3_y)
```

np.loadtxt load data into a 2d array.

np.cov(data1, rowvar=False) calculates the covariance of the data1 data set. Each column represents a variable, and rows contain observations.

np.std(data1[:, 0]) calculates the standard deviation of the first variable (column) in the data1 data set.

8.2 The covariance matrix calculated for each experiments

```
p1 cov:
 [[ 0.00042651 -0.00018885]
 [-0.00018885  0.0006476 ]]
p2 cov:
 [[ 7.11023996e-04 -4.08749897e-05]
 [-4.08749897e-05  6.07364235e-04]]
p3 cov:
 [[ 0.00806129 -0.0029002 ]
 [-0.0029002   0.00449381]]
```

For sanity check

```
p1 std devs:  [0.02065212 0.02544799]
p2 std devs:  [0.02666503 0.02464476]
p3 std devs:  [0.08978469 0.0670359 ]
p1 std dev x:  0.01959231920925525
p1 std dev y:  0.024142084781585523
p2 std dev x:  0.025296671648003848
p2 std dev y:  0.023380072953812857
p3 std dev x:  0.0851772368621729
p3 std dev y:  0.06359583861249941
```

8.3 Discuss possible ways to make scatter smaller for a given level of noise

- Using kalman filter, it provides the estimation of states from noisy environment
- Using low pass filter
- Using signal correction, ignore the very abnormal data
- Increase the signal-to-noise ratio (SNR)

Part IV
Lab 4

9 Task 4-1:

9.1 Code snippets of your implementation of the PD controller

```
class Controller:
    def __init__(self, P=0.0, D=0.0, set_point=0):
        self.Kp = P
        self.Kd = D
        self.set_point = set_point  # reference (desired value)
        self.previous_error = 0

    def update(self, current_value):
        # calculate P_term and D_term
        error = self.set_point - current_value
        P_term = self.Kp * error
        D_term = self.Kd * (error - self.previous_error)
        self.previous_error = error
        return P_term + D_term

    def setPoint(self, set_point):
        self.set_point = set_point
        self.previous_error = 0

    def setPD(self, P=0.0, D=0.0):
        self.Kp = P
        self.Kd = D
```

9.2 Describe your approach and explain your code

I'm using the formula provided by lab

$$u(t) = k_p e(t) + k_d \left(e\left(t\right) - e\left(t - \triangle t\right)\right)$$

- u(t): Is the output of controller, so it is return of P_term+D_term
- Kp & Kd: Getting by setPD, coefficient of the proportional term and derivative term
- e(t): This is the error of system at this time, which is error

- $e\left(t-\triangle t\right)$: previous_error, error for previous stage

So for the code I implement

error = self.set_point - current_value which is expected point minus current value

P_term = self.Kp * error which is relating to $k_p e(t)$ oefficient of the proportional term times error

D_term = self.Kd * (error - self.previous_error) relating to $k_d\left(e\left(t\right)-e\left(t-\triangle t\right)\right)$coefficient of derivative term times (error - previous_error)

self.previous_error = error store the error value

Then return P_term+D_term, that is u(t)

10 Task 4-2:

Create project

```
ros2 pkg create --build-type ament_python lab42
```

Build project by

```
colcon build --packages-select lab42
source install/setup.bash
```

run

```
ros2 run lab42 move
```

10.1 Code snippets of your implementation to drive the Turtlebot using PD control

I'm using PD control for angle only, I init the controller at the __init__ part of class turtlebot3. Using Kp = 0.38 and Kd = 0.03 after several testing. These sets of parameter work best. Then I calculate the desire angle at each loop, input desired angle as set point. Then using the update value as the angular velocity.

```
class Turtlebot3():
    def __init__(self):
        ...

        # init controller
        self.controller = Controller()
        self.controller.setPD(0.38, 0.03)
```

```
        # initial angle should be same to robot prevent turning at the beginning
        self.controller.setPoint(self.pose.theta)

        try:
            self.run()
        except KeyboardInterrupt:
            print('Interrupted')
        finally:
            # save trajectory to csv file
            np.savetxt('trajectory_close.csv', np.array(self.trajectory), delimiter=',')

            self.node.destroy_node()
            rclpy.shutdown()

    def run(self, angle_count=0):
        # add your code here to adjust your movement based on 2D pose feedback
        msg = Twist()
        for i in range(4 * fwd_time * frq):
            noise = np.random.normal(0, l_noise)
            msg.linear.x = distance / fwd_time + noise
            msg.angular.z = 0.0
            self.vel_pub.publish(msg)  # publish the message

            self.node.get_logger().info('[Translation] Publishing: "%s"' % i)
            self.rate.sleep()  # the code will sleep to keep the frequency, in this case 10Hz

            if (i + 1) % (fwd_time * frq) == 0:
                self.rate.sleep()
                angle_count += 1
                desired_angle = pi / 2 * angle_count
                self.controller.setPoint(desired_angle)
                for _ in range(rot_time * frq):
                    noise = np.random.normal(0, a_noise)
                    msg.linear.x = 0.0
                    # msg.angular.z = angle / rot_time + noise
                    msg.angular.z = self.controller.update(self.pose.theta) + noise
                    self.vel_pub.publish(msg)

```

```
                    self.node.get_logger().info('[Rotation]
                    Publishing: "%s"' % i)
                    self.rate.sleep()

        ...
```

Update setup file

```
    entry_points={
        'console_scripts': [
            'move = lab42.pd:main'
        ],
```

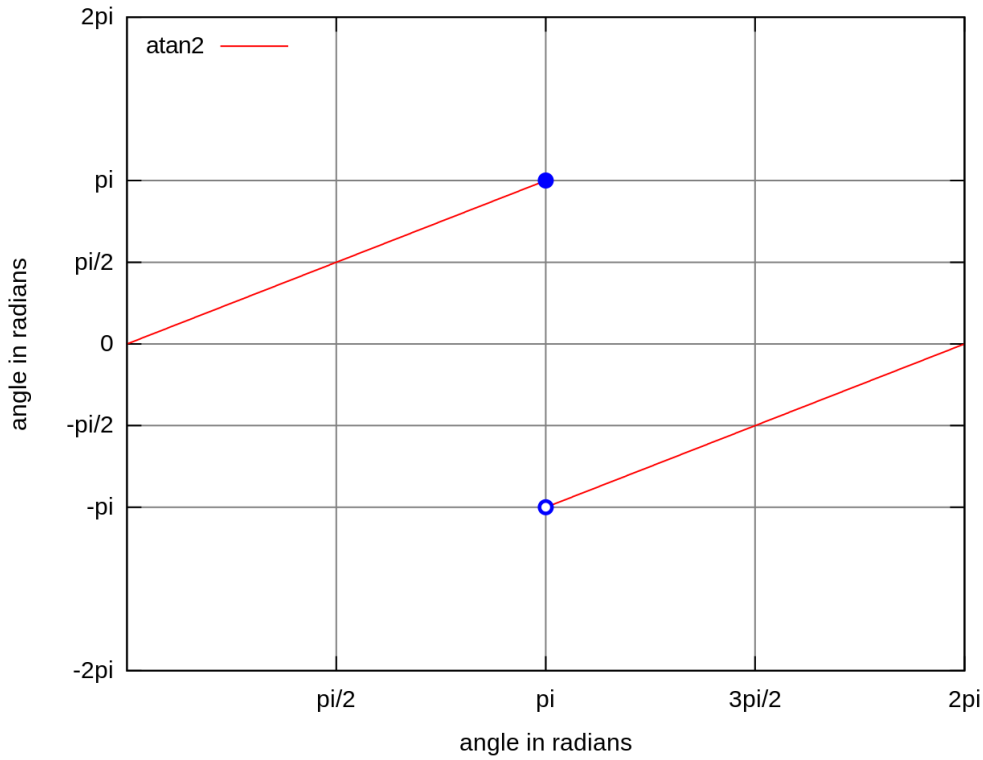
10.1.1 Normalize angle

Because the angle is from -pi to pi, I want to make it to 0 to 2pi ensure code is fine. I add the code inside odom callback

```
    def odom_callback(self, msg):
        # get pose = (x, y, theta) from odometry topic
        quaternion = [msg.pose.pose.orientation.x,
        msg.pose.pose.orientation.y, \
                      msg.pose.pose.orientation.z,
                      msg.pose.pose.orientation.w]
        (roll, pitch, yaw) = euler_from_quaternion(quaternion)
        # Convert yaw to a value between 0 and 2*pi
        yaw = atan2(sin(yaw), cos(yaw))
        if yaw < 0:
            yaw += 2 * pi

        self.pose.theta = yaw
```

By definition, $\vartheta = \text{atan2}\ (\ y\ ,\ x\ )$ is the angle measure (in radians, with $-\pi < \vartheta \leq \pi$ between the positive x-axis and the ray from the origin to the point (x , y) in the Cartesian plane.

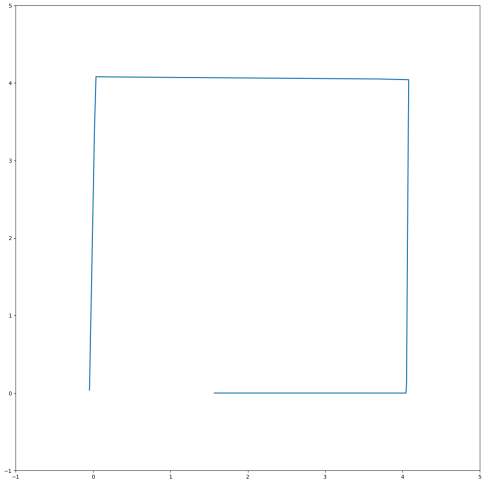


And then add 2pi if angle less then 0.

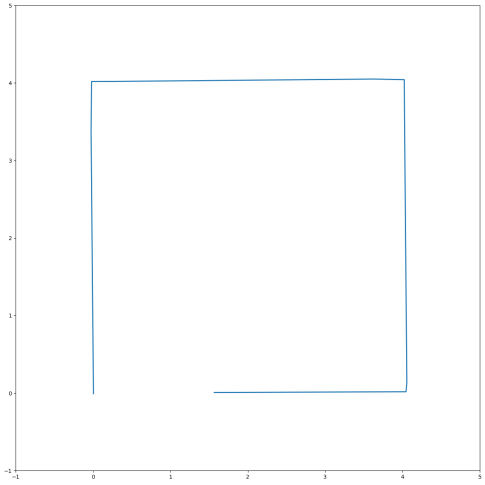
10.2 Plots the trajectories of your robot moving with and without the PD controller.

Using python program provided.

Witout PD



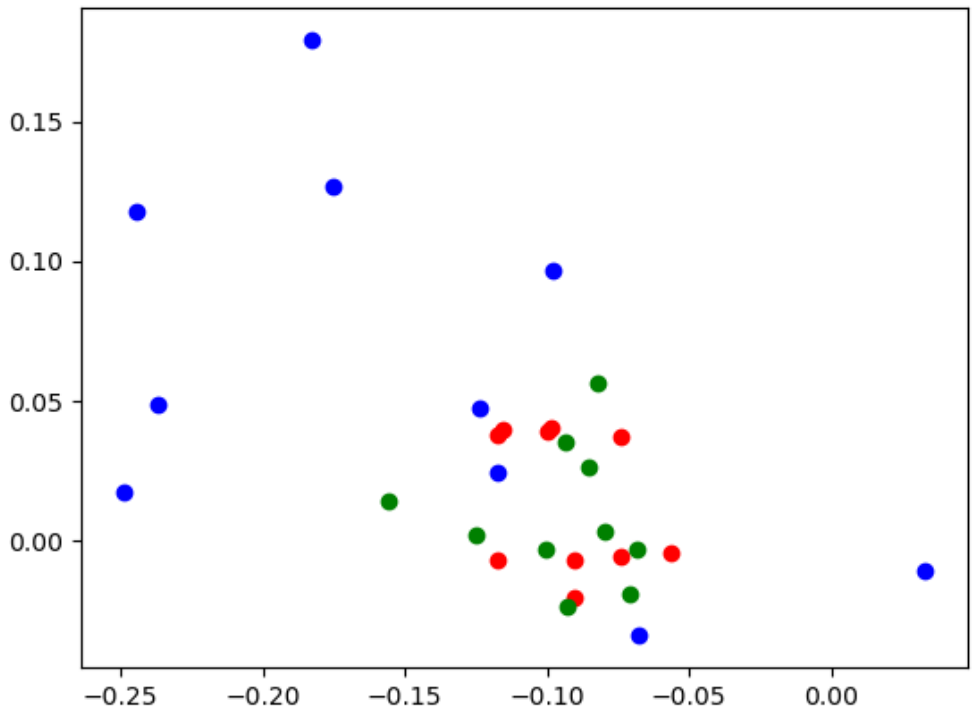
With PD



At no noise level there is not significant difference.

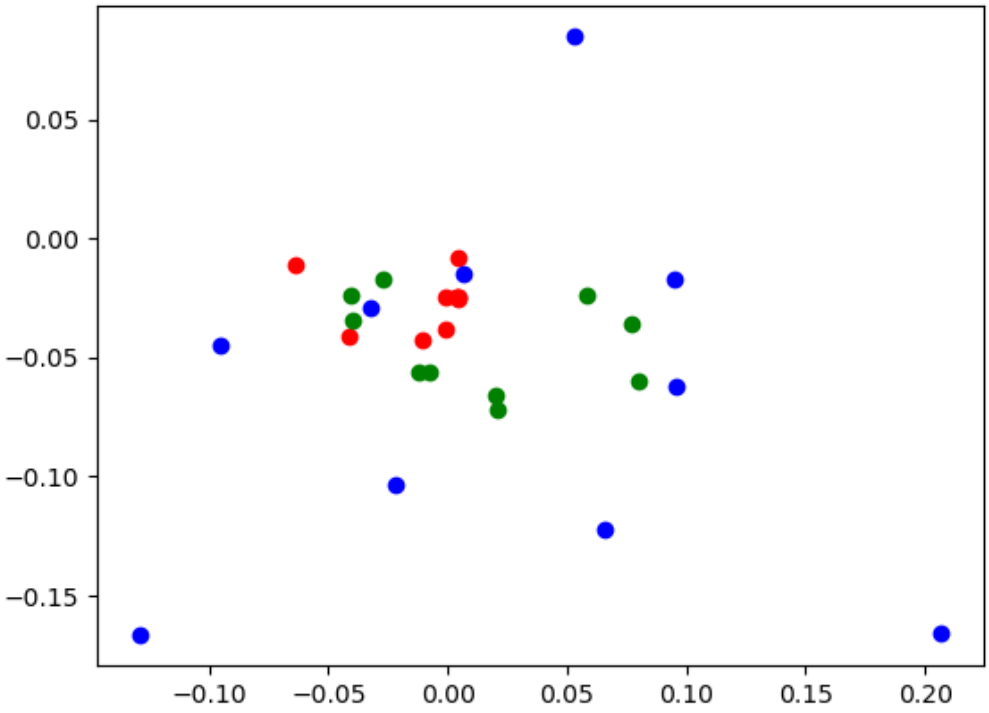
10.3 Scatter plots of the final robot locations ((x,y) coordinates) saved from Lab 3 (open loop control)

For no noise is red, low level is green, high level is blue



10.4 Scatter plots of the final robot locations ((x,y) coordinates) when closed loop control is used

For no noise is red, low level is green, high level is blue



10.5 Reflect your lab activities and discuss the questions

10.5.1 Which causes a larger effect in your robot, uncertainty in drive distance or rotation angle?

I think is rotation angle, because the wrong angle will make larger error when the robot go in straight line. Because the errors in rotation can lead to the robot moving in a completely wrong direction. And then will make error bigger when moving in straight line.

10.5.2 Under different levels of motion noise, how open loop and closed loop control behave?

In a low-noise environment, closed-loop control do better than open-loop control. That is becuase it can correct error by colsed loop control. However, when noise levels are high, both control methods are affected.

10.5.3 Can you think of any scenario where even closed loop control may not work?

A environment with very hight noise this will cause closed loop signal not reliable.

10.5.4 Can you think of any robot designs which would be able to move more precisely?

Higher precision sensors can help mitigate noise, providing more accurate position and orientation information.

10.5.5 How should we go about equipping a robot to recover from the motion drift?

Use visual, tactile, or other environmental feedback to adjust the robot's path in real-time.

Part V
Lab 5

11 Task 5-1:

Create project

```
ros2 pkg create --build-type ament_python lab51
```

Using
Build project by

```
colcon build --packages-select lab51
source install/setup.bash
```

Start simulation

```
ros2 launch turtlebot3_gazebo turtlebot3_world.launch.py
```

11.1 Code snippets

Using the skeleton as Lab4, and using turtlebot3_drive to drive robot

```python
#!/usr/bin/env python
import threading

import rclpy
from sensor_msgs.msg import LaserScan


class Turtlebot3():
    def __init__(self):
        rclpy.init()
        self.node = rclpy.create_node("turtlebot3_lab51")
        self.node.get_logger().info("Press Ctrl + C to terminate")
        self.logging_counter = 0

        # subscribe to laser scan
        self.laser_sub = self.node.create_subscription(LaserScan, "scan", self.laser_callback, 10)

        t = threading.Thread(target=rclpy.spin, args=(self.node,), daemon=True)
        t.start()

        try:
            while True:
                pass
        except KeyboardInterrupt:
            print('Interrupted')
        finally:
            self.node.destroy_node()
            rclpy.shutdown()

    def laser_callback(self, msg):
        # print the distance to the first obstacle in front of the robot
        front = msg.ranges[0]
        left = msg.ranges[89]
        back = msg.ranges[179]
        right = msg.ranges[269]

        # logging once every 25 times (Gazebo runs at 1000Hz; we save it at 10Hz)
        self.logging_counter += 1
        if self.logging_counter == 25:
```

```
            self.logging_counter = 0
            self.node.get_logger().info(f"Front: {front:.4f},
            Left: {left:.4f}, Back: {back:.4f}, Right:
            {right:.4f}")



def main(args=None):
    turtlebot = Turtlebot3()




if __name__ == '__main__':
    main()
```

I subsribe to topic scan, get distance message and print it using logger. Modify setup.py

```
entry_points={
        'console_scripts': [
            'go = lab51.sense:main',
        ],
```

After

```
 ros2 run lab51 go
```

Run turtlebot3_drive in different terminal, and get output like

```
[INFO] [1711570532.175830675] [turtlebot3_lab51]: Press Ctrl + C
to terminate
[INFO] [1711570537.351165529] [turtlebot3_lab51]: Front: 2.0960,
Left: 3.3601, Back: 1.5313, Right: 0.5760
[INFO] [1711570542.422475068] [turtlebot3_lab51]: Front: 2.4848,
Left: 1.2413, Back: 0.9974, Right: 0.4783
[INFO] [171157Create project

Using
Build project by
0547.367546964] [turtlebot3_lab51]: Front: 1.5629, Left: 1.8679,
Back: 1.7484, Right: 0.5705
[INFO] [1711570552.464862782] [turtlebot3_lab51]: Front: 2.7494,
Left: inf, Back: 1.4495, Right: 0.8031
```

12 Task 5-2:

12.1 Code snippets of your implementation for obstacle avoidance

12.1.1 Open loop

```
#!/usr/bin/env python
import threading

import rclpy
from geometry_msgs.msg import Twist
from sensor_msgs.msg import LaserScan

# distance between the robot and the obstacle that should stop
distance = 0.3

class Turtlebot3():
    def __init__(self):
        rclpy.init()
        self.node = rclpy.create_node("turtlebot3_lab52")
        self.node.get_logger().info("Press Ctrl + C to
        terminate")
        self.logging_counter = 0
        self.rate = self.node.create_rate(10)


        # move the robot
        self.cmd_vel_pub = self.node.create_publisher(Twist,
        "cmd_vel", 10)

        # subscribe to laser scan
        self.laser_sub =
        self.node.create_subscription(LaserScan, "scan",
        self.laser_callback, 10)

        # init distance
        self.front = float('inf')
        self.left = float('inf')
        self.back = float('inf')
        self.right = float('inf')

```

```
        t = threading.Thread(target=rclpy.spin,
        args=(self.node,), daemon=True)
        t.start()

        try:
            while True:
                self.run()
        except KeyboardInterrupt:
            print('Interrupted')
        finally:
            self.node.destroy_node()
            rclpy.shutdown()

    def run(self):
        # move the robot forward
        msg = Twist()
        if self.front > distance:
            msg.linear.x = 0.2
        else:
            # stop the robot
            msg.linear.x = 0.0
        self.cmd_vel_pub.publish(msg)
        self.node.get_logger().info(' Robot speed: "%s"' %
        msg.linear.x)
        self.rate.sleep()

    def laser_callback(self, msg):
        # print the distance to the first obstacle in front of
        the robot
        self.front = msg.ranges[0]
        self.left = msg.ranges[89]
        self.back = msg.ranges[179]
        self.right = msg.ranges[269]

        # logging once every 25 times (Gazebo runs at 1000Hz; we
        save it at 10Hz)
        self.logging_counter += 1
        if self.logging_counter == 5:
            self.logging_counter = 0
            self.node.get_logger().info(f"Front:
            {self.front:.4f}, Left: {self.left:.4f}, Back:
            {self.back:.4f}, Right: {self.right:.4f}")
```

```
def main(args=None):
    turtlebot = Turtlebot3()



if __name__ == '__main__':
    main()
```

12.1.2 Closed loop

```
#!/usr/bin/env python
import threading

import rclpy
from geometry_msgs.msg import Twist
from sensor_msgs.msg import LaserScan

# distance between the robot and the obstacle that should stop
distance = 0.3

class Turtlebot3():
    def __init__(self):
        rclpy.init()
        self.node = rclpy.create_node("turtlebot3_lab52")
        self.node.get_logger().info("Press Ctrl + C to
        terminate")
        self.logging_counter = 0
        self.rate = self.node.create_rate(10)

        # init controller
        self.controller = Controller()
        self.controller.setPD(0.5, 0.1)
        # initial angle should be same to robot prevent turning
        at the beginning
        self.controller.setPoint(distance)


        # move the robot
        self.cmd_vel_pub = self.node.create_publisher(Twist,
        "cmd_vel", 10)

        # subscribe to laser scan
        self.laser_sub =
        self.node.create_subscription(LaserScan, "scan",
        self.laser_callback, 10)
```

```
31
32          # init distance
33          self.front = float(3)
34          self.left = float('inf')
35          self.back = float('inf')
36          self.right = float('inf')
37          # keep list of front distance, using 5 readings
38          self.front_list = []
39
40          t = threading.Thread(target=rclpy.spin,
            args=(self.node,), daemon=True)
41          t.start()
42
43          try:
44              while True:
45                  self.run()
46          except KeyboardInterrupt:
47              print('Interrupted')
48          finally:
49              self.node.destroy_node()
50              rclpy.shutdown()
51
52      def run(self):
53          # speed need to be positive
54          speed = self.controller.update(self.front) * -1
55          # move the robot forward
56          msg = Twist()
57          # control robot only move forward
58          msg.linear.x = min(speed, 1.0)
59
60          self.cmd_vel_pub.publish(msg)
61          self.node.get_logger().info(' Robot speed: "%s"' %
            speed)
62          self.rate.sleep()
63
64      def laser_callback(self, msg):
65          # append new reading
66          self.front_list.append(msg.ranges[0])
67          if len(self.front_list) > 5:
68              # keep only 5 readings
69              self.front_list.pop(0)
70          self.front = sum(self.front_list) / len(self.front_list)
71          self.left = msg.ranges[89]
72          self.back = msg.ranges[179]
73          self.right = msg.ranges[269]
```

```
74
75          # logging once every 25 times (Gazebo runs at 1000Hz; we
            save it at 10Hz)
76          self.logging_counter += 1
77          if self.logging_counter == 5:
78              self.logging_counter = 0
79              self.node.get_logger().info(f"Front:
                {self.front:.4f}, Left: {self.left:.4f}, Back:
                {self.back:.4f}, Right: {self.right:.4f}")
80
81
82  class Controller:
83          ...
84          # same as lab 4
85
86
87  def main(args=None):
88      turtlebot = Turtlebot3()
89
90
91
92  if __name__ == '__main__':
93      main()
```

12.2 Clearly describe your approach and explain your code including gain tuning

12.2.1 Open loop

This robot will stop at distance less than 0.3, create a publisher that public twist message. If the distance is larger than 0.3, set speed to 0.2 or set speed to 0. Using self.front to store the distance of front.

12.2.2 Closed loop

Init of front distance change to 3 to prevent calculation error. Using the controller as same as the implementation of lab 4, because the error is negative but we need positive to move robot forward, so times -1 to change the direction. Set max speed to 1 prevent the speed is too high at some point. At the run(), update current value as front distance.

In laser_callback, using a list to keep last 5 readings of front distance. Calculate the mean as the front distance to get smooth operation.

At beginning set kp and kd in small number like 0.1, 0.01 and gradually increse the number to get the best performance. Now I'm using kp = 0.5 and kd = 0.1.

12.3 Clear discussion on possible ways of dealing with non-zero velocity of the obstacles

As the example of keeping it at a safe distance from the car in front on a motorway, it requires some different strategies. For instance, to maintain a safe distance from the vehicle ahead, the system needs to react quickly to sudden movements, which means higher monitoring frequency. Also, predicting the motion of obstacles, especially moving ones, is essential, and linear prediction methods can help.

Also we could use PID controller instead PD controller. The integral term in PID controllers is better at handling accumulated errors, resulting in improved control.

13 Task 5-3:

Create package

```
ros2 pkg create --build-type ament_python lab53
```

Build project by

```
colcon build --packages-select lab53
source install/setup.bash
```

Run it by

```
ros2 run lab53 follow
```

13.1 Code snippets of your implementation for wall following.

```
#!/usr/bin/env python
import threading

import rclpy
from math import pi
from geometry_msgs.msg import Twist
from sensor_msgs.msg import LaserScan
import numpy as np

# distance between the robot and the obstacle that should stop
distance = 0.3
```

```
frq = 10  # frequency: Hz
rotation_time = 3
rot_angle = pi / 2.0
a_noise = 0.0
l_noise = 0.0

class Turtlebot3():
    def __init__(self):
        rclpy.init()
        self.node = rclpy.create_node("turtlebot3_lab53")
        self.node.get_logger().info("Press Ctrl + C to
        terminate")
        self.logging_counter = 0
        self.rate = self.node.create_rate(10)

        # init controller
        self.controller = Controller()
        self.controller.setPD(0.6, 0.1)
        self.controller.setPoint(distance)

        # controller to follow wall
        self.controller_wall = Controller()
        self.controller_wall.setPoint(0.3)
        self.controller_wall.setPD(0.2, 0.1)
        # flag to show if robot foward to desired distance
        self.forward_flag = False


        # move the robot
        self.cmd_vel_pub = self.node.create_publisher(Twist,
        "cmd_vel", 10)

        # subscribe to laser scan
        self.laser_sub =
        self.node.create_subscription(LaserScan, "scan",
        self.laser_callback, 10)

        # init distance
        self.front = float(3)
        self.left = float('inf')
        self.back = float('inf')
        self.right = float('inf')
        # keep list of front distance, using 5 readings
        self.front_list = []
        self.right_list = []
```

```
53
54          t = threading.Thread(target=rclpy.spin,
            args=(self.node,), daemon=True)
55          t.start()
56
57          try:
58              while not self.forward_flag:
59                  self.run_forward()
60              self.node.get_logger().info(' Robot reach the
                desired distance')
61              self.turn_left()
62              while True:
63                  self.follow_wall()
64          except KeyboardInterrupt:
65              print('Interrupted')
66          finally:
67              self.node.destroy_node()
68              rclpy.shutdown()
69
70      def run_forward(self):
71          # speed need to be positive
72          speed = self.controller.update(self.front) * -1
73          # move the robot forward
74          msg = Twist()
75          # control robot only move forward
76          msg.linear.x = min(speed, 1.0)
77
78          self.cmd_vel_pub.publish(msg)
79          # check if robot reach the desired distance
80          if self.front < distance and speed < 0:
81              self.forward_flag = True
82              self.node.get_logger().info(' Robot reach the
                desired distance')
83              msg.linear.x = 0.0
84              self.cmd_vel_pub.publish(msg)
85          # self.node.get_logger().info(' Robot speed: "%s"' %
            speed)
86          self.rate.sleep()
87
88      def turn_left(self):
89          # just code to rotate the robot 90 degree
90          msg = Twist()
91          for _ in range(rotation_time * frq):
92              msg.angular.z = rot_angle / rotation_time
93              msg.linear.x = 0.0
```

```
94              self.cmd_vel_pub.publish(msg)
95              self.node.get_logger().info(' Robot speed: "%s"' %
                msg.angular.z)
96              self.rate.sleep()
97          msg.angular.z = 0.0
98          self.cmd_vel_pub.publish(msg)
99
100     def follow_wall(self):
101         msg = Twist()
102         angular_speed = self.controller_wall.update(self.right)
103         if self.front_range < distance:
104                 # time to turn left, stop robot first
105             msg.linear.x = 0.0
106             msg.angular.z = 0.0
107             self.cmd_vel_pub.publish(msg)
108             self.node.get_logger().info(' Start turning left')
109             for _ in range(rotation_time * frq):
110                 msg.angular.z = rot_angle / rotation_time +
                    np.random.normal(0, a_noise)
111                 msg.linear.x = 0.0
112                 self.cmd_vel_pub.publish(msg)
113                 self.node.get_logger().info(' Robot speed: "%s"'
                    % msg.angular.z)
114                 self.rate.sleep()
115             self.node.get_logger().info('Finished turning left')
116         else:
117             msg.angular.z = angular_speed + np.random.normal(0,
                a_noise)
118             msg.linear.x = 0.15 + np.random.normal(0, l_noise)
119             self.cmd_vel_pub.publish(msg)
120             self.node.get_logger().info(' Robot speed: "%s",
                with right distance "%s"' % (msg.angular.z,
                self.right))
121         self.rate.sleep()
122
123     def laser_callback(self, msg):
124         # append new reading
125         self.front_list.append(msg.ranges[0])
126         if len(self.front_list) > 3:
127             # keep only 3 readings
128             self.front_list.pop(0)
129         self.front = sum(self.front_list) / len(self.front_list)
130         self.left = msg.ranges[89]
131         self.back = msg.ranges[179]
132
```

```
133         self.right =
            (msg.ranges[240]+msg.ranges[269]+msg.ranges[300])/3
134         self.front_range =
            (msg.ranges[330]+msg.ranges[0]+msg.ranges[30])/3
135
136         # logging once every 25 times (Gazebo runs at 1000Hz; we
            save it at 10Hz)
137         self.logging_counter += 1
138         if self.logging_counter == 5:
139             self.logging_counter = 0
140             self.node.get_logger().info(f"Front:
                {self.front:.4f}, Left: {self.left:.4f}, Back:
                {self.back:.4f}, Right: {self.right:.4f}")
141
142 def main(args=None):
143     turtlebot = Turtlebot3()
144
145 if __name__ == '__main__':
146     main()
```

13.2 Clearly describe your approach and explain your code.

This task is divided into three parts. Firstly, the robot should move directly to the wall until it reaches a distance of 0.3m. Then, it should turn left 90 degrees using closed control for angular speed. Next, the robot should move linearly for a distance of 0.15m and utilize PD control to maintain a distance of 0.3m from the wall on its right side. If there is a wall within 0.3m, the robot should stop and turn left again. I use the average range obtained from three directions instead of detecting only the front and right directions. This is because if the robot is at some angle, the reading may not accurately represent the distance between the robot and the wall. Utilizing the average can improve performance. For now, the robot can successfully navigate around with distances ranging from 0.2m to 0.4m.

13.3 How different levels of motion noise may impact the wall following behavior of the robot. And possible way to achieve smooth wall following behavior when significant motion noise presents.

As the noise level increases, the system becomes less stable. At low noise levels (around 0.001 to 0.01), the system can operate normally, but the robot's movements may become more erratic and distances less precise. However, when noise reaches 0.1, the robot's large deviation angle makes it impossible for the PD system to correct, causing the robot to fail in executing the task.

We could use filter like kalman filter on the data,and ignore the too abnormal data.

Using more robust control system, like PID instead of PD. Adding an integral term that eliminate steady-state error. Or reduce the P term so the system won't be so sensitive to noise.